Docker Challenge

Create a customized Docker image using a Dockerfile.

Create an instance, install docker and git and clone the repository

```
[root@ip-10-0-1-206 ~]# git clone https://github.com/betawins/Python-app.git
Cloning into 'Python-app'...
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 25 (delta 10), reused 4 (delta 4), pack-reused 10 (from 1)
Receiving objects: 100% (25/25), 6.79 KiB | 6.79 MiB/s, done.
Resolving deltas: 100% (10/10), done.
```

```
[root@ip-10-0-1-206 ~]# ls
Python-app
[root@ip-10-0-1-206 ~]# cd Python-app/
[root@ip-10-0-1-206 Python-app]# ls -la
total 20
drwxr-xr-x. 4 root root  128 Oct 24 18:41 .
dr-xr-x---. 4 root root  121 Oct 24 18:41 ..
drwxr-xr-x. 7 root root  147 Oct 24 18:41 .git
-rw-r--r--. 1 root root 1799 Oct 24 18:41 .gitignore
-rw-r--r--. 1 root root  112 Oct 24 18:41 Dockerfile
-rw-r--r--. 1 root root  975 Oct 24 18:41 Jenkinsfile
-rw-r--r--. 1 root root 2258 Oct 24 18:41 app.py
-rw-r--r--. 1 root root    5 Oct 24 18:41 requirements.txt
drwxr-xr-x. 2 root root   24 Oct 24 18:41 templates
[root@ip-10-0-1-206 Python-app]#
```

```
ENTRYPOINT ["python", "app.py"][rcat > Dockerfile <<'EOF'-app]# cat > Dockerfile <<'EOF'
FROM python:3.11-slim

WORKDIR /opt

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

CMD ["python", "app.py"]
EOF
```

```
[root@ip-10-0-1-206 Python-app]# cat Dockerfile
FROM python:3.11-slim

WORKDIR /opt

COPY requirements.txt .

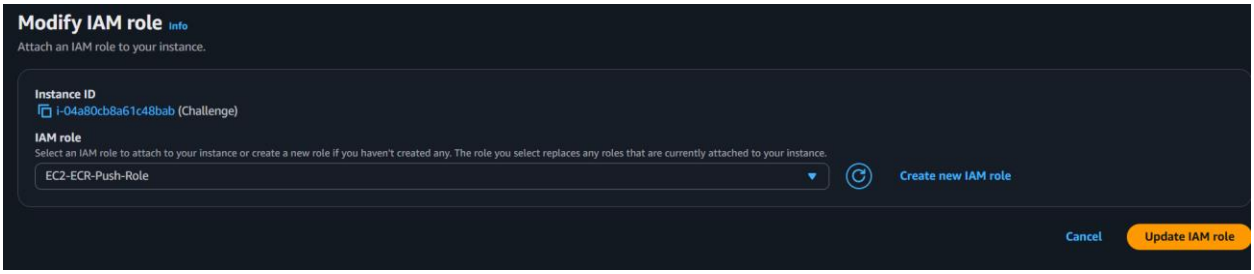
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8080

CMD ["python", "app.py"]
```

```
[root@ip-10-0-1-206 Python-app]# docker build -t python-app:v1 .
[+] Building 4.8s (10/10) FINISHED                                                
```




```
[root@ip-10-0-1-206 Python-app]# aws sts get-caller-identity
{
    "UserId": "AROAW2OTU2QGRSUTC3255:i-04a80cb8a61c48bab",
    "Account": "469132170253",
    "Arn": "arn:aws:sts::469132170253:assumed-role/EC2-ECR-Push-Role/i-04a80cb8a61c48bab"
}
```

IAM attached

```
[root@ip-10-0-1-206 Python-app]# aws ecr create-repository \
  --repository-name python-app \
  --region us-east-1
{
    "repository": {
        "repositoryArn": "arn:aws:ecr:us-east-1:469132170253:repository/python-app",
        "registryId": "469132170253",
        "repositoryName": "python-app",
        "repositoryUri": "469132170253.dkr.ecr.us-east-1.amazonaws.com/python-app",
        "createdAt": "2026-10-24T19:05:12.200000+00:00",
        "imageTagMutability": "MUTABLE",
        "imageScanningConfiguration": {
            "scanOnPush": false
        },
        "encryptionConfiguration": {
            "encryptionType": "AES256"
        }
    }
}
```

Ecr repository created

```
[root@ip-10-0-1-206 Python-app]# aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 469132170253.dkr.ecr.us-east-1.amazon
m
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

login to ecr is successful

```
[root@ip-10-0-1-206 Python-app]# docker tag python-app:v1 469132170253.dkr.ecr.us-east-1.amazonaws.com/python-app:v1
[root@ip-10-0-1-206 Python-app]# docker push 469132170253.dkr.ecr.us-east-1.amazonaws.com/python-app:v1
The push refers to repository [469132170253.dkr.ecr.us-east-1.amazonaws.com/python-app]
ee6dcd716aa1: Pushed
faf028a5cb8a: Pushed
d8cbb291da95: Pushed
5f70bf18a086: Pushed
d86e9bd00483: Pushed
6972161a9a77: Pushed
535a5d38abec: Pushed
a70f6272827f: Pushed
v1: digest: sha256:5d5673e315d7de9d053b2c036c6b255c0d103999e8ac88304a167162b56fef81 size: 1992
[root@ip-10-0-1-206 Python-app]# aws ecr describe-images --repository-name python-app --region us-east-1
{
    "imageDetails": [
        {
            "registryId": "469132170253",
            "repositoryName": "python-app",
            "imageDigest": "sha256:5d5673e315d7de9d053b2c036c6b255c0d103999e8ac88304a167162b56fef81",
            "imageTags": [
                "v1"
            ],
            "imageSizeInBytes": 52374068,
            "imagePushedAt": "2026-10-24T19:07:52.046000+00:00",
            "imageManifestMediaType": "application/vnd.docker.distribution.manifest.v2+json",
            "artifactMediaType": "application/vnd.docker.container.image.v1+json",
            "imageStatus": "ACTIVE"
        }
    ]
}
[root@ip-10-0-1-206 Python-app]#
```

tag and push the image to ecr

```
[root@ip-10-0-1-206 Python-app]# dnf install -y yum-utils
Last metadata expiration check: 0:30:36 ago on Sat Oct 24 18:38:42 2026.
Package dnf-utils-4.3.0-13.amzn2023.0.7.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-1-206 Python-app]# yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
Adding repo from: https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
[root@ip-10-0-1-206 Python-app]# dnf install -y terraform
Hashicorp Stable - x86_64                                                                    52 MB/s | 1.9 MB     00:00
Dependencies resolved.
================================================================================================================================
 Package                         Architecture                    Version                         Repository                   Si
================================================================================================================================
Installing:
 terraform                       x86_64                          1.17.0-1                        hashicorp                    37

Transaction Summary
================================================================================================================================
```

install yum-utils and then add the official Hashicorp repository and install terraform

```
[root@ip-10-0-1-206 Python-app]# terraform --version
Terraform v1.17.0
on linux_amd64

[root@ip-10-0-1-206 Python-app]# mkdir -p /root/terraform-jenkins
cd /root/terraform-jenkins
[root@ip-10-0-1-206 terraform-jenkins]# aws sts get-caller-identity
{
    "UserId": "AROAW2OTU2QGRSUTC3255:i-04a80cb8a61c48bab",
    "Account": "469132170253",
    "Arn": "arn:aws:sts::469132170253:assumed-role/EC2-ECR-Push-Role/i-04a80cb8a61c48bab"
}
[root@ip-10-0-1-206 terraform-jenkins]#
```

Create a terraform configuration

cat > main.tf <<'EOF'

provider "aws" {

region = "us-east-1"

}

resource "aws_security_group" "jenkins_sg" {

name = "jenkins-server-sg"

description = "Allow SSH and Jenkins access"

ingress {

description = "SSH"

from_port = 22

to_port = 22

protocol = "tcp"

cidr_blocks = ["0.0.0.0/0"]

}

ingress {

description = "Jenkins"

```
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "jenkins" {
  ami                    = "ami-08b5b3a93ed654d19"
  instance_type          = "t3.small"
  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  user_data = <<-USERDATA
    #!/bin/bash
    dnf update -y
    dnf install -y java-21-amazon-corretto wget
    wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
    rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
    dnf install -y jenkins
    systemctl enable jenkins
    systemctl start jenkins
  USERDATA
```

```
  tags = {
    Name = "Jenkins-Server"
  }
}

output "jenkins_public_ip" {
  value = aws_instance.jenkins.public_ip
}
EOF
```

This uses an Amazon Linux 2023 AMI ID for us-east-1; if AWS rejects it, we'll look up the correct AMI rather than guessing further. Port 8080 is temporarily open to the internet for this practice project. For better security, restrict SSH and Jenkins access to your own IP.

```
[root@ip-10-0-1-206 terraform-jenkins]# terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.70.0...
- Installed hashicorp/aws v6.70.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-10-0-1-206 terraform-jenkins]#
```

```
[root@ip-10-0-1-206 terraform-jenkins]# terraform validate
Success! The configuration is valid.
```

```
[root@ip-10-0-1-206 terraform-jenkins]# terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.jenkins will be created
  + resource "aws_instance" "jenkins" {
      + ami                                  = "ami-08b5b3a93ed654d19"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.small"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
```

Terraform will show what AWS resources it plans to create, including:

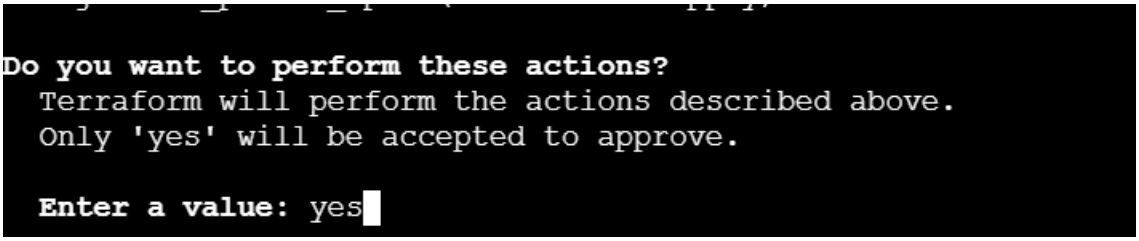
One security group for Jenkins

One EC2 instance (t3.small)

A startup script to install Java and Jenkins

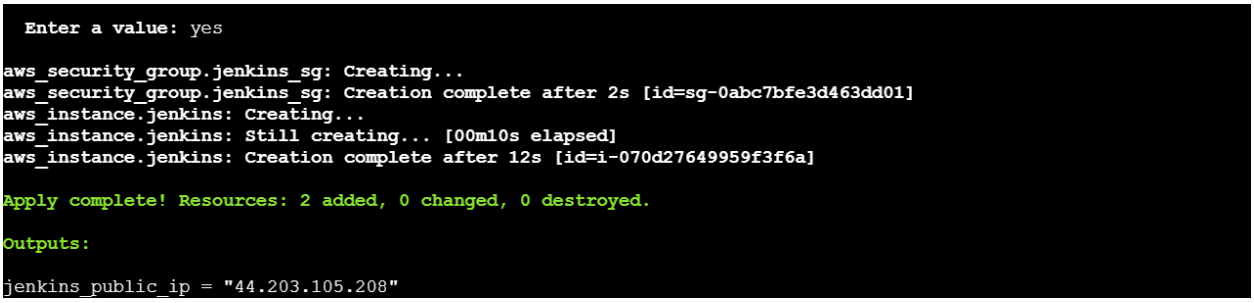
We see like-------Plan: 2 to add, 0 to change, 0 to destroy.

That means Terraform plans to create the Jenkins security group and EC2 instance.

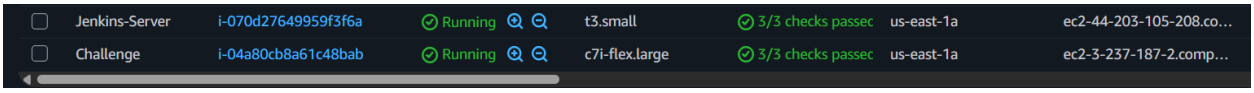




So we need to attach full EC2 acess

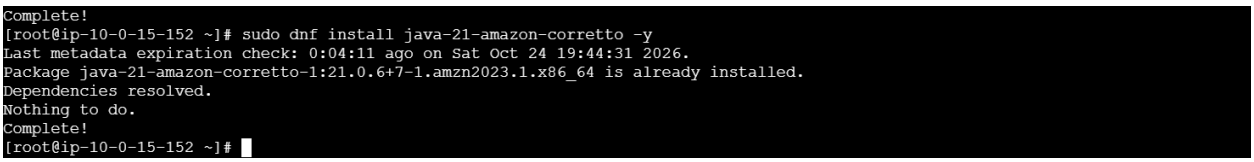


Terraform resources are created



Jenkins server created

Now connect to new instance, run

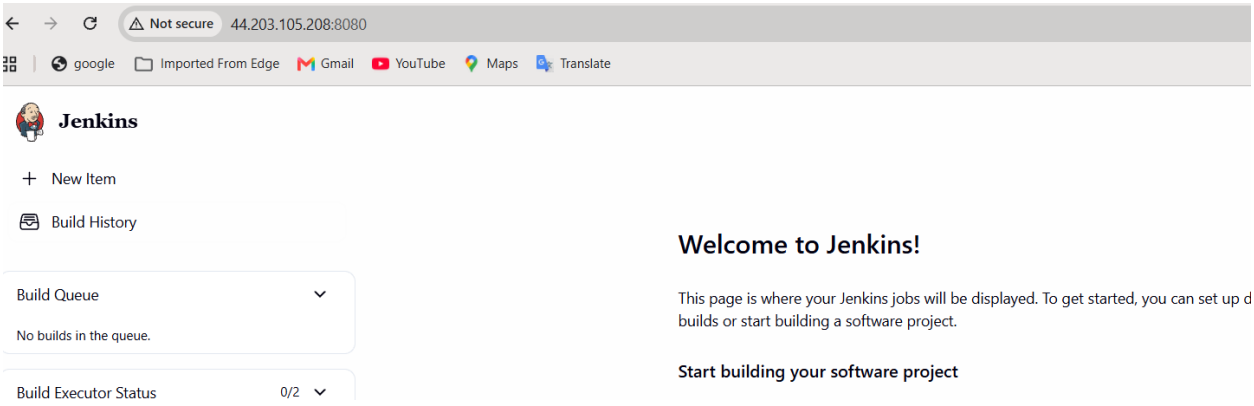


Then install Jenkins

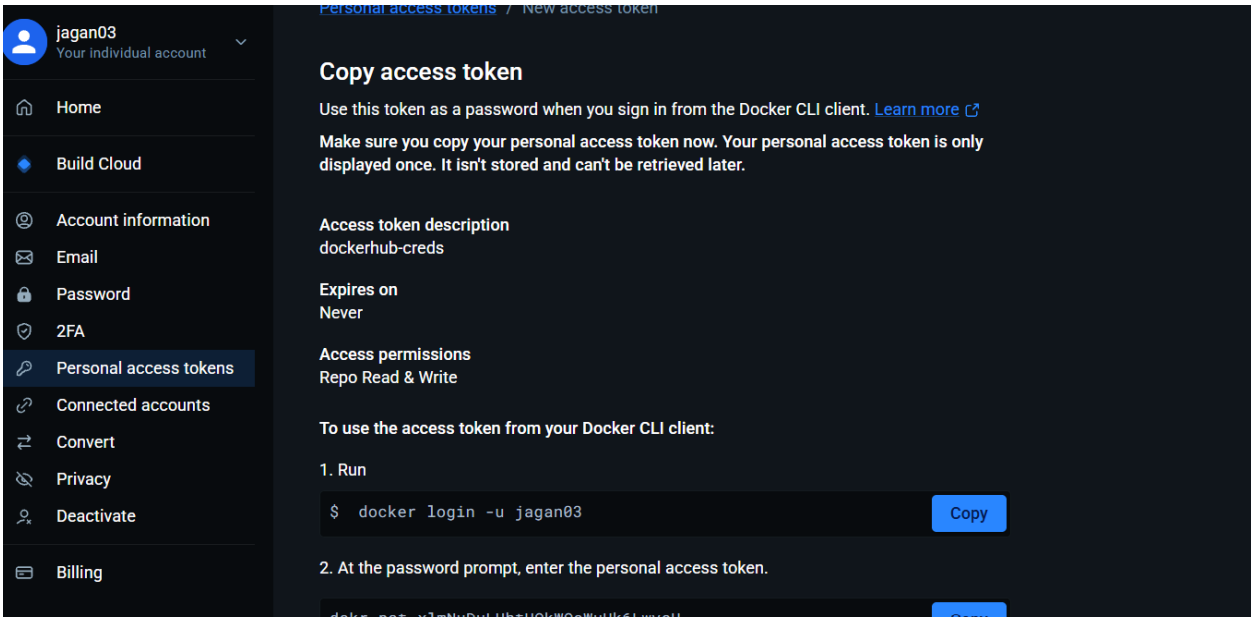
```
[root@ip-10-0-15-152 ~]# dnf install jenkins -y
Last metadata expiration check: 0:05:42 ago on Sat Oct 24 19:44:31 2026.
Package jenkins-2.580.1-1.noarch is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-15-152 ~]#
```

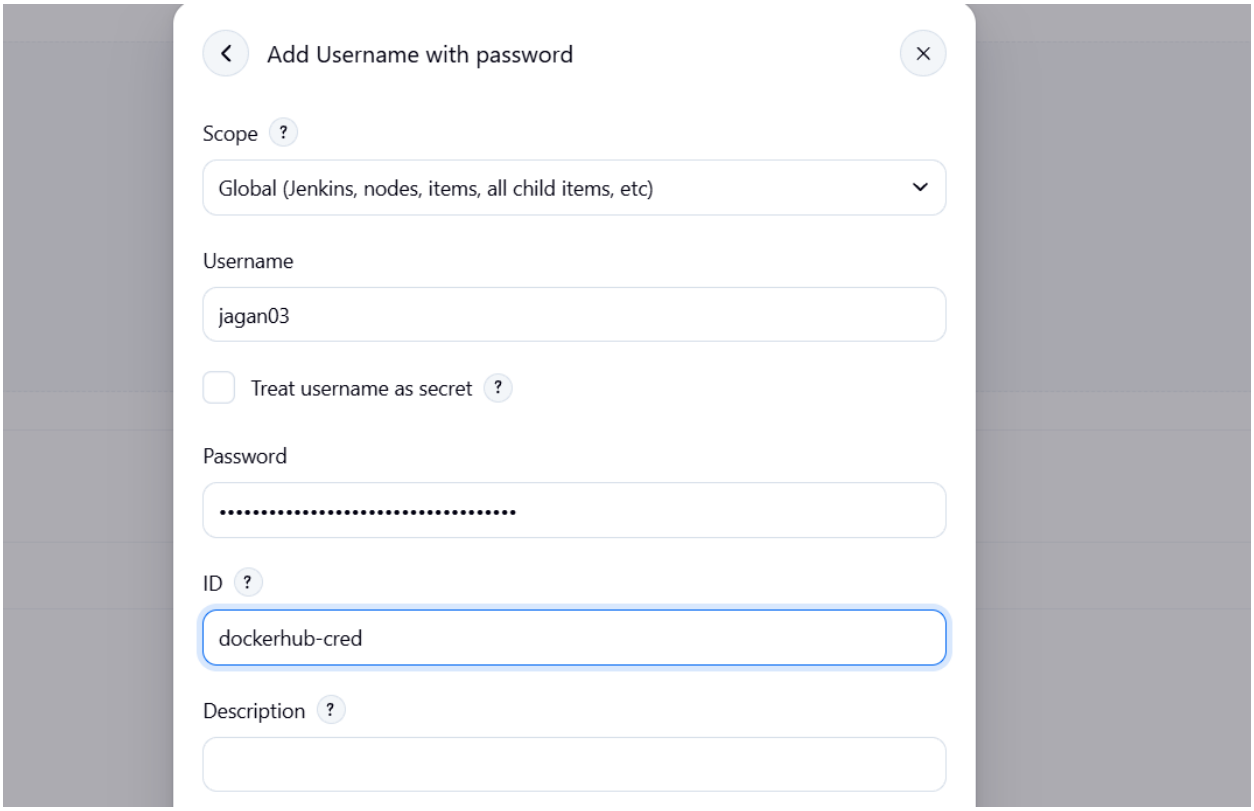
```
● jenkins.service - Jenkins Continuous Integration Server
     Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
     Active: active (running) since Sat 2026-10-24 19:44:53 UTC; 5min ago
   Main PID: 13084 (java)
      Tasks: 43 (limit: 2253)
```

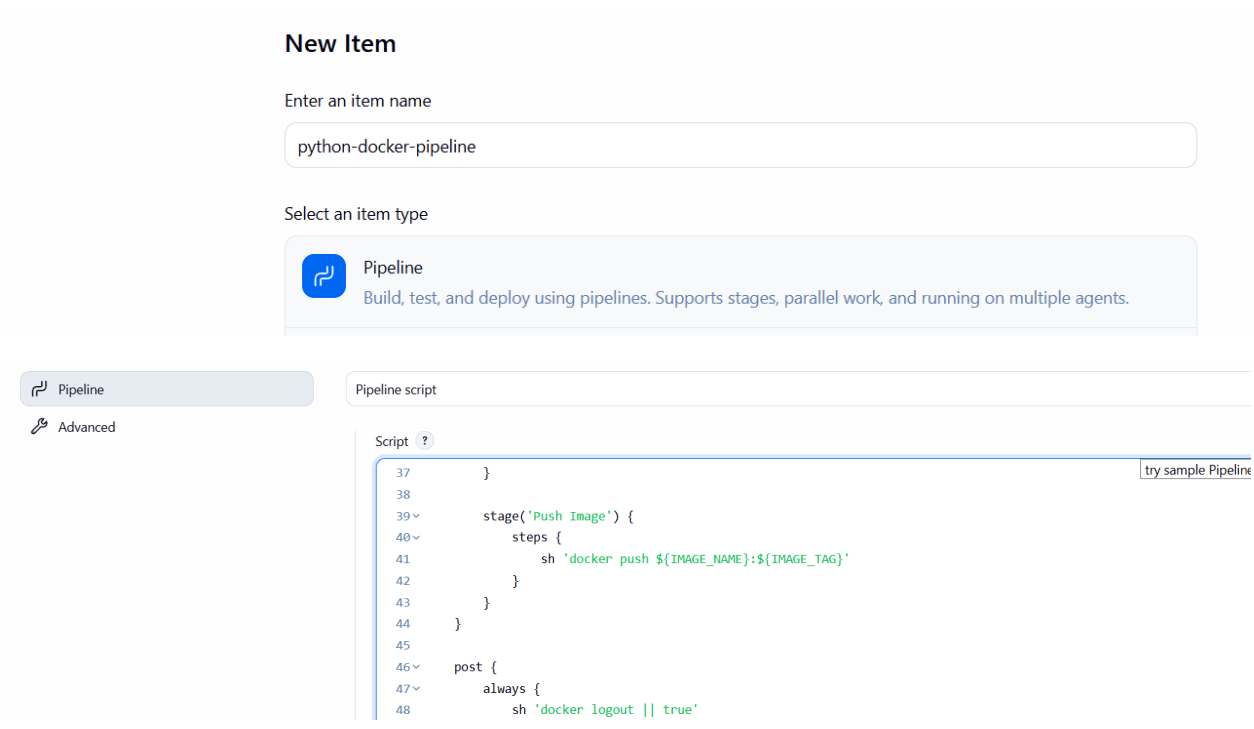
```
[root@ip-10-0-15-152 ~]# sudo -u jenkins docker --version
Docker version 25.0.8, build 0bab007
[root@ip-10-0-15-152 ~]#
```



Verify can Jenkins can run docker







add the script in the pipeline

This pipeline clones your code, builds the Docker image, logs in using the saved credential, and pushes jagan03/python-app:v1 to Docker Hub.

```
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```